

LAPORAN TUGAS 3

SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Mochamad Rio Aprilian (2410651040)

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

BAB 1

PENDAHULUAN

1.1 LATAR BELAKANG

Perkembangan teknologi komputasi modern menuntut pemanfaatan sumber daya sistem secara efisien, salah satunya melalui pemrograman paralel. Pemrograman paralel memungkinkan eksekusi beberapa tugas secara bersamaan, yang dapat meningkatkan kinerja aplikasi secara signifikan. Dalam sistem operasi, konsep multithreading dan sinkronisasi proses menjadi fondasi penting untuk mengoptimalkan penggunaan prosesor dan memori. Thread memungkinkan pembagian tugas dalam sebuah proses, sedangkan mekanisme sinkronisasi seperti mutex dan buffer bersama diperlukan untuk mengelola akses data secara aman. Melalui praktikum ini, mahasiswa diajak untuk memahami implementasi konkurensi dalam bahasa C menggunakan library pthread, yang merupakan dasar dalam pengembangan perangkat lunak yang efisien dan responsif. Pemahaman ini tidak hanya berguna dalam konteks akademis tetapi juga dalam pengembangan aplikasi dunia nyata yang memerlukan penanganan banyak tugas secara simultan.

1.2 RUMUSAN MASALAH

1. Bagaimana mengimplementasikan kalkulator paralel menggunakan multithreading untuk menjalankan operasi aritmatika secara bersamaan?
2. Bagaimana membandingkan kinerja antara pendekatan multithread dan singlethread dalam pemrosesan teks?
3. Bagaimana mengelola sinkronisasi antara producer dan consumer dalam mengakses buffer bersama untuk menghindari race condition?

3.1 TUJUAN

1. Membuat program kalkulator paralel yang dapat menjalankan operasi penjumlahan, pengurangan, perkalian, dan pembagian secara bersamaan menggunakan thread.
2. Menganalisis perbedaan efisiensi waktu antara pemrosesan teks dengan multithread dan singlethread.
3. Mengimplementasikan solusi producer-consumer dengan sinkronisasi yang aman menggunakan mutex dan buffer melingkar.

BAB II

PEMBAHASAN

2.1 PRAKTIK TUGAS 1

Program kalkulator paralel ini merupakan implementasi konkurensi menggunakan thread dalam bahasa C yang memanfaatkan library pthread. Kode ini menjalankan empat operasi aritmatika dasar (penjumlahan, pengurangan, perkalian, dan pembagian) secara bersamaan dengan membagi setiap operasi ke dalam thread terpisah. Setiap thread berjalan paralel untuk menghitung operasinya masing-masing menggunakan data input yang sama dari pengguna, yang disimpan dalam struktur data KalkulatorData yang berisi dua angka input, hasil perhitungan, dan string operasi. Mekanisme program dimulai dengan pengambilan input dua angka dari user, kemudian membuat empat struktur data terpisah untuk setiap operasi guna menghindari race condition. Program menggunakan pthread_create untuk membuat dan menjalankan setiap thread dengan fungsi yang sesuai, lalu pthread_join untuk menunggu semua thread menyelesaikan komputasinya sebelum menampilkan hasil akhir. Pendekatan ini memungkinkan eksekusi yang lebih efisien karena perhitungan dilakukan secara paralel, meskipun untuk operasi sederhana seperti ini overhead thread management mungkin lebih dominan dibanding manfaat paralelisasinya. Program juga telah mengantisipasi error pada pembagian dengan nol dengan pengecekan eksplisit, serta menggunakan array karakter tetap daripada pointer untuk menyimpan nama operasi guna menghindari masalah memori. Hasil perhitungan dari setiap thread ditampilkan secara real-time saat eksekusi dan kemudian dirangkum dalam output akhir, menunjukkan kemampuan program dalam mengelola resource bersama dan sinkronisasi thread dengan efektif.

2.1.1 Membuat Kalkulator Paralel

```
rio_regex@DESKTOP-NVPMHQ:~$ nano kalkulator_paralel.c
rio_regex@DESKTOP-NVPMHQ:~$ gcc kalkulator_paralel.c -o kalkulator_paralel -pthread
rio_regex@DESKTOP-NVPMHQ:~$ ./kalkulator_paralel
=== KALKULATOR PARALEL ===
Masukkan angka pertama: 25
Masukkan angka kedua: 10

Memulai perhitungan paralel...

Thread Penjumlahan: 25.00 + 10.00 = 35.00
Thread Pengurangan: 25.00 - 10.00 = 15.00
Thread Perkalian: 25.00 * 10.00 = 250.00
Thread Pembagian: 25.00 / 10.00 = 2.50

=== HASIL AKHIR ===
Penjumlahan: 35.00
Pengurangan: 15.00
Perkalian: 250.00
Pembagian: 2.50
rio_regex@DESKTOP-NVPMHQ:~$ |
```

2.1.2 Penejelasan Proses Kompilasi dan Eksekusi

A. PROSES KOMPILASI:

- Preprocessing: File kode sumber diproses oleh preprocessor untuk memasukkan header files seperti stdio.h dan pthread.h ke dalam kode

- **Compilation:** Kode C dikompilasi menjadi assembly language menggunakan GCC compiler dengan perintah `gcc -o kalkulator kalkulator.c -lpthread`
- **Assembly:** Kode assembly diubah menjadi object code dalam format binary yang bisa dimengerti mesin
- **Linking:** Object code di-link dengan pthread library untuk mendukung fungsi multithreading yang digunakan program
- **Output:** Menghasilkan file executable kalkulator yang bisa langsung dijalankan di terminal

B. PROSES EKSEKUSI:

- **Program Start:** File executable diload ke memory dan proses utama mulai berjalan
- **User Input:** Program meminta pengguna memasukkan dua angka yang akan dihitung melalui keyboard
- **Data Setup:** Membuat empat variabel struktur data terpisah untuk menyimpan input dan hasil setiap operasi
- **Thread Preparation:** Menyiapkan empat thread yang akan menjalankan operasi matematika berbeda-beda

C. PARALEL PROCESSING:

- **Thread Launch:** Keempat thread dijalankan bersamaan menggunakan `pthread_create()`
- **Concurrent Calculations:** Masing-masing thread melakukan perhitungan secara paralel:
 - Thread 1: Melakukan penjumlahan dua angka
 - Thread 2: Melakukan pengurangan dua angka
 - Thread 3: Melakukan perkalian dua angka
 - Thread 4: Melakukan pembagian dengan pengecekan pembagi nol
- **Real-time Output:** Setiap thread menampilkan hasil sementara begitu perhitungannya selesai

D. RESULTS COLLECTION:

- **Thread Synchronization:** Program utama menunggu semua thread menyelesaikan tugasnya dengan `pthread_join()`
- **Data Aggregation:** Mengumpulkan hasil akhir dari semua struktur data yang telah diupdate oleh thread-thread
- **Final Display:** Menampilkan ringkasan semua operasi matematika dalam format yang rapi dan mudah dibaca
- **Program Exit:** Mengakhiri program dengan membersihkan semua resources yang digunakan

E. KEUNIKAN PROGRAM:

- Independent Execution: Setiap thread bekerja mandiri dengan data sendiri-sendiri sehingga tidak bentrok
- Error Handling: Thread pembagian punya fitur deteksi error khusus untuk pembagian dengan nol
- Parallel Demo: Meski perhitungannya sederhana, program ini menunjukkan konsep parallel computing dengan jelas
- User Friendly: Tampilan outputnya bertahap sehingga mudah dipahami alur eksekusinya

F. Kode C:

```
#include <stdio.h>

#include <pthread.h>

#include <stdlib.h>

#include <string.h> // dibutuhkan untuk strcpy

// Struktur untuk menyimpan data yang dibagikan ke thread
typedef struct {

    double angka1;

    double angka2;

    double hasil;

    char operasi[20]; // ubah dari pointer jadi array agar aman

} KalkulatorData;

// Fungsi untuk penjumlahan
void* tambah(void* arg) {

    KalkulatorData* data = (KalkulatorData*) arg;

    data->hasil = data->angka1 + data->angka2;

    strcpy(data->operasi, "Penjumlahan");

    printf("Thread Penjumlahan: %.2lf + %.2lf = %.2lf\n",

        data->angka1, data->angka2, data->hasil);

    return NULL;

}

// Fungsi untuk pengurangan
void* kurang(void* arg) {
```

```

KalkulatorData* data = (KalkulatorData*) arg;

data->hasil = data->angka1 - data->angka2; // tambahkan tanda minus

strcpy(data->operasi, "Pengurangan");

printf("Thread Pengurangan: %.2lf - %.2lf = %.2lf\n",

      data->angka1, data->angka2, data->hasil);

return NULL;
}

// Fungsi untuk perkalian
void* kali(void* arg) {

    KalkulatorData* data = (KalkulatorData*) arg;

    data->hasil = data->angka1 * data->angka2;

    strcpy(data->operasi, "Perkalian");

    printf("Thread Perkalian: %.2lf * %.2lf = %.2lf\n",

          data->angka1, data->angka2, data->hasil);

    return NULL;
}

// Fungsi untuk pembagian
void* bagi(void* arg) {

    KalkulatorData* data = (KalkulatorData*) arg;

    strcpy(data->operasi, "Pembagian");

    if (data->angka2 != 0) {

        data->hasil = data->angka1 / data->angka2;

        printf("Thread Pembagian: %.2lf / %.2lf = %.2lf\n",

              data->angka1, data->angka2, data->hasil);

    } else {

        data->hasil = 0;

        printf("Thread Pembagian: Error! Pembagian dengan nol.\n");

    }

    return NULL;
}

```

```

}

int main() {
    double angka1, angka2;

    // Input dari user
    printf("=== KALKULATOR PARALEL ===\n");
    printf("Masukkan angka pertama: ");
    scanf("%lf", &angka1);
    printf("Masukkan angka kedua: ");
    scanf("%lf", &angka2);
    printf("\n");

    // Deklarasi thread dan data
    pthread_t thread_tambah, thread_kurang, thread_kali, thread_bagi;
    KalkulatorData data_tambah, data_kurang, data_kali, data_bagi;

    // Inisialisasi data
    data_tambah.angka1 = data_kurang.angka1 = data_kali.angka1 = data_bagi.angka1 =
    angka1;
    data_tambah.angka2 = data_kurang.angka2 = data_kali.angka2 = data_bagi.angka2 =
    angka2;

    printf("Memulai perhitungan paralel...\n\n");

    // Membuat thread
    pthread_create(&thread_tambah, NULL, tambah, &data_tambah);
    pthread_create(&thread_kurang, NULL, kurang, &data_kurang);
    pthread_create(&thread_kali, NULL, kali, &data_kali);
    pthread_create(&thread_bagi, NULL, bagi, &data_bagi);

    // Menunggu thread selesai
    pthread_join(thread_tambah, NULL);
    pthread_join(thread_kurang, NULL);
    pthread_join(thread_kali, NULL);
    pthread_join(thread_bagi, NULL);

```

```

// Hasil akhir

printf("\n=== HASIL AKHIR ===\n");

printf("%s: %.2lf\n", data_tambah.operasi, data_tambah.hasil);

printf("%s: %.2lf\n", data_kurang.operasi, data_kurang.hasil);

printf("%s: %.2lf\n", data_kali.operasi, data_kali.hasil);

printf("%s: %.2lf\n", data_bagi.operasi, data_bagi.hasil);

return 0;

}

```

2.2 PRAKTIK TUGAS 2

Program ini merupakan implementasi penghitungan kata dalam file teks menggunakan pendekatan paralel dengan multithreading dibandingkan dengan versi singlethread. Kode menggunakan library pthread untuk membuat dua thread yang bekerja secara bersamaan, masing-masing menghitung jumlah kata pada separuh bagian teks yang berbeda. Program dimulai dengan membuat file contoh otomatis, kemudian membaca seluruh konten file ke dalam memori untuk diproses lebih lanjut oleh kedua thread. Algoritma pembagian kerja didesain dengan mempertimbangkan integritas kata, dimana titik pembagian teks dicari hingga menemukan spasi atau newline agar tidak memotong kata di tengah. Setiap thread menjalankan fungsi `hitung\_kata\_thread` yang menggunakan finite state machine untuk melacak transisi antara spasi dan karakter kata. Program juga mengukur waktu eksekusi kedua versi menggunakan `clock()` untuk perbandingan performa, menunjukkan efektivitas paralelisasi pada teks berukuran cukup besar. Hasil perhitungan dari kedua thread kemudian digabungkan, dan program memberikan analisis komparatif mengenai mana yang lebih cepat antara pendekatan multithreading dan singlethread. Meskipun untuk teks kecil overhead pembuatan thread mungkin membuat singlethread lebih cepat, program ini mendemonstrasikan konsep fundamental dalam pemrograman paralel dan manajemen thread untuk optimalisasi performa processing teks.

2.2.1 Membuat File Processing

- Otomatis


```

rio_regex@DESKTOP-NVPMHQ:~$ nano file_processing.c
rio_regex@DESKTOP-NVPMHQ:~$ gcc file_processing.c -o file_processing -pthread
rio_regex@DESKTOP-NVPMHQ:~$ ./file_processing
File contoh.txt berhasil dibuat
=== FILE PROCESSING PARALEL ===
Panjang teks: 311 karakter

=== VERSI MULTITHREAD ===
Thread: Menghitung kata dari indeks 0-157 = 24 kata
Thread: Menghitung kata dari indeks 157-311 = 19 kata
Total kata (multithread): 43
Waktu eksekusi: 0.000542 detik

=== VERSI SINGLETHREAD ===
Total kata (singlethread): 43
Waktu eksekusi: 0.000005 detik

=== PERBANDINGAN ===
Multithread: 43 kata, 0.000542 detik
Singlethread: 43 kata, 0.000005 detik
KESIMPULAN: Singlethread lebih cepat 0.000537 detik
rio_regex@DESKTOP-NVPMHQ:~$

```

• Manual Menggunakan input.txt

```

rio_regex@DESKTOP-NVPMHQ:~$ nano input.txt
rio_regex@DESKTOP-NVPMHQ:~$ nano file_processing.c
rio_regex@DESKTOP-NVPMHQ:~$ gcc file_processing.c -o file_processing -pthread
rio_regex@DESKTOP-NVPMHQ:~$ ./file_processing
File contoh.txt berhasil dibuat
=== FILE PROCESSING PARALEL ===
Panjang teks: 311 karakter

=== VERSI MULTITHREAD ===
Thread: Menghitung kata dari indeks 0-157 = 24 kata
Thread: Menghitung kata dari indeks 157-311 = 19 kata
Total kata (multithread): 43
Waktu eksekusi: 0.000594 detik

=== VERSI SINGLETHREAD ===
Total kata (singlethread): 43
Waktu eksekusi: 0.000004 detik

=== PERBANDINGAN ===
Multithread: 43 kata, 0.000594 detik
Singlethread: 43 kata, 0.000004 detik
KESIMPULAN: Singlethread lebih cepat 0.000590 detik
rio_regex@DESKTOP-NVPMHQ:~$

```

2.2.2 Penjelasan Proses Kompilasi dan Eksekusi

A. PROSES KOMPILASI:

- Preprocessing: File kode sumber diproses untuk memasukkan semua header file seperti stdio.h dan pthread.h ke dalam kode
- Compilation: Kode C dikompilasi menjadi assembly language menggunakan GCC compiler dengan tambahan opsi -lpthread
- Assembly: Kode assembly diubah menjadi object code dalam format binary yang bisa dimengerti processor
- Linking: Object code di-link dengan pthread library dan library standar C untuk menghasilkan executable
- Output: Terbentuk file executable yang siap dijalankan di sistem operasi

B. PROSES EKSEKUSI:

- Program Start: File executable diload ke memory dan proses utama mulai berjalan
- File Creation: Program membuat file contoh.txt secara otomatis dengan konten teks yang sudah ditentukan
- File Reading: Membaca seluruh isi file contoh.txt ke dalam memory menggunakan alokasi dynamic
- Data Preparation: Menghitung panjang teks dan menentukan titik pembagian untuk dua thread

C. MULTITHREAD PROCESSING:

- Thread Initialization: Membuat dua thread yang akan bekerja paralel menghitung kata
- Smart Partitioning: Membagi teks menjadi dua bagian dengan mencari batas kata yang tepat agar tidak memotong kata
- Parallel Execution: Dua thread berjalan bersamaan menghitung kata di bagian masing-masing:
 - Thread 1: Memproses bagian awal teks dari indeks 0 sampai titik tengah
 - Thread 2: Memproses bagian akhir teks dari titik tengah sampai akhir
- Real-time Output: Setiap thread menampilkan progress dan hasil perhitungannya

D. PERFORMANCE COMPARISON:

- Multithread Timing: Mencatat waktu mulai dan selesai untuk versi multithread
- Singlethread Benchmark: Menjalankan penghitungan kata dengan single thread untuk perbandingan
- Data Combination: Menjumlahkan hasil dari kedua thread untuk mendapatkan total kata
- Result Analysis: Membandingkan waktu eksekusi kedua metode dan menentukan mana yang lebih cepat

E. CLEANUP & TERMINATION:

- Memory Management: Membebaskan memory yang dialokasikan untuk menyimpan teks file
- Thread Cleanup: Sistem operasi mengatur pembersihan resources thread yang sudah selesai
- Program Completion: Menampilkan kesimpulan akhir dan mengakhiri program dengan status sukses

F. Kode C:

```
#include <stdio.h>

#include <pthread.h>

#include <stdlib.h>

#include <string.h>

#include <time.h>

#include <sys/stat.h>

// Struktur untuk data thread

typedef struct {

    char* text;
```

```

    int start;

    int end;

    int word_count;
} ThreadData;

// Fungsi menghitung kata dalam bagian teks

int hitung_kata_bagian(char* text, int start, int end) {

    int count = 0;

    int in_word = 0;

    for (int i = start; i < end; i++) {

        if (text[i] == ' ' || text[i] == '\n' || text[i] == '\t' || text[i] == '\0') {

            if (in_word) {

                count++;

                in_word = 0;

            }

        } else {

            in_word = 1;

        }

    }

    // Hitung kata terakhir jika teks berakhir tanpa spasi

    if (in_word) {

        count++;

    }

    return count;

}

// Fungsi thread untuk menghitung kata

void* hitung_kata_thread(void* arg) {

    ThreadData* data = (ThreadData*) arg;

    data->word_count = hitung_kata_bagian(data->text, data->start, data->end);

```

```

printf("Thread: Menghitung kata dari indeks %d-%d = %d kata\n",
      data->start, data->end, data->word_count);

return NULL;
}

// Fungsi singlethread (versi biasa)
int hitung_kata_singlethread(char* text) {
    return hitung_kata_bagian(text, 0, strlen(text));
}

// Fungsi baca file
char* baca_file(const char* filename) {
    FILE* file = fopen(filename, "r");
    if (!file) {
        printf("Error: Tidak bisa membuka file %s\n", filename);
        return NULL;
    }

    // Cari ukuran file
    fseek(file, 0, SEEK_END);
    long size = ftell(file);
    fseek(file, 0, SEEK_SET);

    // Allokasi memori untuk teks
    char* text = malloc(size + 1);
    fread(text, 1, size, file);
    text[size] = '\0';
    fclose(file);
    return text;
}

// Fungsi buat file contoh
void buat_file_contoh() {
    FILE* file = fopen("contoh.txt", "w");

```

```

if (file) {
    fprintf(file, "Ini adalah file teks contoh untuk praktikum sistem operasi\n");
    fprintf(file, "Kita akan menghitung jumlah kata dalam file ini\n");
    fprintf(file, "Menggunakan dua thread yang bekerja paralel\n");
    fprintf(file, "Thread pertama menghitung separuh bagian awal\n");
    fprintf(file, "Thread kedua menghitung separuh bagian akhir\n");
    fprintf(file, "Hasilnya akan digabungkan dan dibandingkan dengan versi singlethread\n");
    fclose(file);

    printf("File contoh.txt berhasil dibuat\n");
}
}

int main() {
    // Buat file contoh otomatis
    buat_file_contoh();

    // Baca file
    char* text = baca_file("contoh.txt");

    if (!text) {
        return 1;
    }

    int text_length = strlen(text);

    printf("=== FILE PROCESSING PARALEL ===\n");
    printf("Panjang teks: %d karakter\n\n", text_length);

    // Versi MULTITHREAD
    printf("=== VERSI MULTITHREAD ===\n");

    clock_t start_multi = clock();

    pthread_t thread1, thread2;
    ThreadData data1, data2;

    // Bagi teks menjadi 2 bagian
    int mid = text_length / 2;

```

```

// Pastikan pembagian tidak memotong kata di tengah
while (mid < text_length && text[mid] != ' ' && text[mid] != '\n') {
    mid++;
}

data1.text = text;
data1.start = 0;
data1.end = mid;
data2.text = text;
data2.start = mid;
data2.end = text_length;

// Buat thread
pthread_create(&thread1, NULL, hitung_kata_thread, &data1);
pthread_create(&thread2, NULL, hitung_kata_thread, &data2);

// Tunggu thread selesai
pthread_join(thread1, NULL);
pthread_join(thread2, NULL);

// Gabungkan hasil
int total_multi = data1.word_count + data2.word_count;
clock_t end_multi = clock();

double waktu_multi = (double)(end_multi - start_multi) / CLOCKS_PER_SEC;
printf("Total kata (multithread): %d\n", total_multi);
printf("Waktu eksekusi: %.6f detik\n\n", waktu_multi);

// Versi SINGLETHREAD
printf("=== VERSI SINGLETHREAD ===\n");
clock_t start_single = clock();

int total_single = hitung_kata_singlethread(text);
clock_t end_single = clock();

double waktu_single = (double)(end_single - start_single) / CLOCKS_PER_SEC;
printf("Total kata (singlethread): %d\n", total_single);

```

```

printf("Waktu eksekusi: %.6f detik\n\n", waktu_single);

// Perbandingan

printf("=== PERBANDINGAN ===\n");

printf("Multithread: %d kata, %.6f detik\n", total_multi, waktu_multi);

printf("Singlethread: %d kata, %.6f detik\n", total_single, waktu_single);

if (waktu_multi < waktu_single) {

    printf("KESIMPULAN: Multithread lebih cepat %.6f detik\n",

        waktu_single - waktu_multi);

} else {

    printf("KESIMPULAN: Singlethread lebih cepat %.6f detik\n",

        waktu_multi - waktu_single);

}

// Bersihkan memori

free(text);

return 0;

}

```

2.3 PRAKTIK TUGAS 3

Program ini mengimplementasikan solusi klasik masalah Producer-Consumer menggunakan thread dan mutex dalam bahasa C. Kode mensimulasikan dua entitas yang berbagi buffer terbatas berukuran 10 slot, dimana producer thread bertugas menghasilkan item berupa angka random dan memasukkannya ke buffer, sementara consumer thread mengambil item tersebut dan memprosesnya dengan mengkuadratkan nilainya. Mekanisme sinkronisasi critical section dijamin menggunakan pthread mutex yang mengontrol akses eksklusif ke buffer bersama, mencegah race condition saat modifikasi data. Algoritma circular buffer diterapkan dengan variabel in dan out yang mengelola penempatan dan pengambilan item secara bergiliran, memastikan utilisasi buffer yang optimal. Producer bekerja lebih cepat dengan delay 100ms dibanding consumer yang sengaja diperlambat menjadi 150ms untuk mensimulasikan ketidakseimbangan kecepatan proses. Program juga dilengkapi fungsi monitoring yang menampilkan status buffer secara real-time, termasuk jumlah item, posisi pointer, dan isi keseluruhan buffer, memberikan visualisasi jelas tentang dinamika interaksi antara producer dan consumer. Simulasi berjalan hingga 20 item diproses, mendemonstrasikan bagaimana mekanisme waiting diterapkan ketika buffer penuh atau kosong, serta efektivitas mutex dalam mengkoordinasikan akses bersama. Implementasi ini menjadi contoh fundamental dalam

pemrograman konkuren untuk mengelola shared resource secara aman dan efisien antara multiple thread yang berjalan paralel.

2.3.1 Membuat Procedur Consumer

```
rio_regex@DESKTOP-NVPMSHQ:~$ nano producer_consumer.c
rio_regex@DESKTOP-NVPMSHQ:~$ gcc producer_consumer.c -o producer_consumer -pthread
```



```

rio_regex@DESKTOP-NVPMHQ:~$ ./producer_consumer
=== SIMULASI PRODUCER-CONSUMER ===
Buffer size: 10 item
Total item yang akan diproduksi: 20

=== STATUS BUFFER ===
Count: 0/10
Isi buffer: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
In: 0, Out: 0

PRODUCER: Hasilkan item 87 | Buffer: 1/10 item
CONSUMER: Ambil item 87 -> Proses:  $87^2 = 7569$  | Buffer: 0/10 item
PRODUCER: Hasilkan item 48 | Buffer: 1/10 item
CONSUMER: Ambil item 48 -> Proses:  $48^2 = 2304$  | Buffer: 0/10 item
PRODUCER: Hasilkan item 96 | Buffer: 1/10 item
CONSUMER: Ambil item 96 -> Proses:  $96^2 = 9216$  | Buffer: 0/10 item
PRODUCER: Hasilkan item 96 | Buffer: 1/10 item
PRODUCER: Hasilkan item 67 | Buffer: 2/10 item
CONSUMER: Ambil item 96 -> Proses:  $96^2 = 9216$  | Buffer: 1/10 item
PRODUCER: Hasilkan item 47 | Buffer: 2/10 item
CONSUMER: Ambil item 67 -> Proses:  $67^2 = 4489$  | Buffer: 1/10 item
PRODUCER: Hasilkan item 47 | Buffer: 2/10 item
PRODUCER: Hasilkan item 62 | Buffer: 3/10 item
CONSUMER: Ambil item 47 -> Proses:  $47^2 = 2209$  | Buffer: 2/10 item
PRODUCER: Hasilkan item 24 | Buffer: 3/10 item
CONSUMER: Ambil item 47 -> Proses:  $47^2 = 2209$  | Buffer: 2/10 item
PRODUCER: Hasilkan item 41 | Buffer: 3/10 item
PRODUCER: Hasilkan item 7 | Buffer: 4/10 item
CONSUMER: Ambil item 62 -> Proses:  $62^2 = 3844$  | Buffer: 3/10 item
PRODUCER: Hasilkan item 36 | Buffer: 4/10 item
CONSUMER: Ambil item 24 -> Proses:  $24^2 = 576$  | Buffer: 3/10 item
PRODUCER: Hasilkan item 86 | Buffer: 4/10 item
PRODUCER: Hasilkan item 14 | Buffer: 5/10 item
CONSUMER: Ambil item 41 -> Proses:  $41^2 = 1681$  | Buffer: 4/10 item
PRODUCER: Hasilkan item 22 | Buffer: 5/10 item
CONSUMER: Ambil item 7 -> Proses:  $7^2 = 49$  | Buffer: 4/10 item
PRODUCER: Hasilkan item 92 | Buffer: 5/10 item
PRODUCER: Hasilkan item 1 | Buffer: 6/10 item
CONSUMER: Ambil item 36 -> Proses:  $36^2 = 1296$  | Buffer: 5/10 item
PRODUCER: Hasilkan item 82 | Buffer: 6/10 item
CONSUMER: Ambil item 86 -> Proses:  $86^2 = 7396$  | Buffer: 5/10 item
PRODUCER: Hasilkan item 93 | Buffer: 6/10 item
PRODUCER: Hasilkan item 3 | Buffer: 7/10 item
CONSUMER: Ambil item 14 -> Proses:  $14^2 = 196$  | Buffer: 6/10 item
PRODUCER: Selesai memproduksi 20 item
CONSUMER: Ambil item 22 -> Proses:  $22^2 = 484$  | Buffer: 5/10 item
CONSUMER: Ambil item 92 -> Proses:  $92^2 = 8464$  | Buffer: 4/10 item
CONSUMER: Ambil item 1 -> Proses:  $1^2 = 1$  | Buffer: 3/10 item
CONSUMER: Ambil item 82 -> Proses:  $82^2 = 6724$  | Buffer: 2/10 item
CONSUMER: Ambil item 93 -> Proses:  $93^2 = 8649$  | Buffer: 1/10 item
CONSUMER: Ambil item 3 -> Proses:  $3^2 = 9$  | Buffer: 0/10 item
CONSUMER: Selesai memproses 20 item

=== STATUS BUFFER ===
Count: 0/10
Isi buffer: [7, 36, 86, 14, 22, 92, 1, 82, 93, 3]
In: 0, Out: 0

=== SIMULASI SELESAI ===
rio_regex@DESKTOP-NVPMHQ:~$ |

```

2.3.2 Penjelasan Proses Kompilasi dan Eksekusi

A. PROSES KOMPILASI:

- Preprocessing: File kode sumber diproses untuk memasukkan semua header file seperti `stdio.h` dan `pthread.h` ke dalam kode
- Compilation: Kode C dikompilasi menjadi assembly language menggunakan GCC compiler dengan tambahan opsi `-lpthread`
- Assembly: Kode assembly diubah menjadi object code dalam format binary yang bisa dimengerti processor
- Linking: Object code di-link dengan pthread library untuk mendukung fungsi multithreading yang digunakan
- Output: Menghasilkan file executable yang siap dijalankan melalui terminal atau command prompt

B. PERSIAPAN PROGRAM:

- Program Start: Sistem operasi memuat file executable ke memory dan memulai eksekusi
- Initial Setup: Program mengatur generator angka random berdasarkan waktu sistem saat ini
- Buffer Initialization: Mengosongkan buffer bersama dan mengatur semua nilai awal ke nol
- Info Display: Menampilkan pengaturan simulasi seperti ukuran buffer dan total item yang akan diproduksi

C. PROSES PRODUCER:

- Item Generation: Membuat angka random antara 1 sampai 100 sebagai item baru
- Buffer Check: Memeriksa apakah buffer masih ada space kosong untuk item baru
- Item Insertion: Memasukkan item ke dalam buffer pada posisi yang ditunjuk pointer 'in'
- Buffer Update: Menambah counter dan memindahkan pointer 'in' ke posisi berikutnya
- Waiting Time: Beristirahat sebentar 100ms antara produksi item-item berikutnya

D. PROSES CONSUMER:

- Availability Check: Memeriksa apakah buffer berisi item yang bisa diambil
- Item Retrieval: Mengambil item dari buffer pada posisi yang ditunjuk pointer 'out'
- Item Processing: Melakukan proses perhitungan kuadrat pada item yang diambil
- Buffer Update: Mengurangi counter dan memindahkan pointer 'out' ke posisi berikutnya
- Processing Delay: Beristirahat 150ms antara pengambilan item untuk simulasi yang realistis

E. MANAJEMEN SYNCHRONISASI:

- Mutual Exclusion: Menggunakan mutex untuk mengamankan akses ke buffer bersama

- Thread Coordination: Program utama menunggu kedua thread selesai sebelum menutup
- Final Report: Menampilkan status buffer terakhir dan ringkasan hasil simulasi
- Cleanup: Sistem secara otomatis membersihkan resources yang digunakan selama program berjalan

F. Kode C:

```
#include <stdio.h>

#include <pthread.h>

#include <stdlib.h>

#include <unistd.h>

#include <time.h>

#define BUFFER_SIZE 10

#define MAX_ITEM 20

// Struktur buffer bersama
typedef struct {
    int buffer[BUFFER_SIZE];
    int count;
    int in;
    int out;
} Buffer;

Buffer shared_buffer;

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

// Fungsi producer
void* producer(void* arg) {
    int produced = 0;
    while (produced < MAX_ITEM) {
        // Generate angka random
        int item = rand() % 100 + 1;
        pthread_mutex_lock(&mutex);
        // Cek jika buffer penuh
```

```

if (shared_buffer.count < BUFFER_SIZE) {
    shared_buffer.buffer[shared_buffer.in] = item;
    shared_buffer.in = (shared_buffer.in + 1) % BUFFER_SIZE;
    shared_buffer.count++;
    printf("PRODUCER: Hasilkan item %d | Buffer: %d/10 item\n",
        item, shared_buffer.count);
    produced++;
} else {
    printf("PRODUCER: Buffer penuh! Menunggu...\n");
}

pthread_mutex_unlock(&mutex);
usleep(100000); // Tidur 100ms
}

printf("PRODUCER: Selesai memproduksi %d item\n", MAX_ITEM);
return NULL;
}

// Fungsi consumer

void* consumer(void* arg) {
    int consumed = 0;
    while (consumed < MAX_ITEM) {
        pthread_mutex_lock(&mutex);
        // Cek jika buffer kosong
        if (shared_buffer.count > 0) {
            int item = shared_buffer.buffer[shared_buffer.out];
            shared_buffer.out = (shared_buffer.out + 1) % BUFFER_SIZE;
            shared_buffer.count--;
            // Proses item (kuadratkan)
            int processed = item * item;
            printf("CONSUMER: Ambil item %d -> Proses: %d2 = %d | Buffer: %d/10 item\n",

```

```

item, item, processed, shared_buffer.count);
consumed++;
} else {
printf("CONSUMER: Buffer kosong! Menunggu...\n");
}
pthread_mutex_unlock(&mutex);
usleep(150000); // Tidur 150ms (lebih lambat dari producer)
}
printf("CONSUMER: Selesai memproses %d item\n", MAX_ITEM);
return NULL;
}

// Inisialisasi buffer
void init_buffer() {
shared_buffer.count = 0;
shared_buffer.in = 0;
shared_buffer.out = 0;
for (int i = 0; i < BUFFER_SIZE; i++) {
shared_buffer.buffer[i] = 0;
}
}

// Fungsi tampilkan status buffer
void tampilkan_buffer() {
printf("\n=== STATUS BUFFER ===\n");
printf("Count: %d/10\n", shared_buffer.count);
printf("Isi buffer: [");
for (int i = 0; i < BUFFER_SIZE; i++) {
printf("%d", shared_buffer.buffer[i]);
if (i < BUFFER_SIZE - 1) printf(", ");
}
}

```

```

printf("]\n");

printf("In: %d, Out: %d\n\n", shared_buffer.in, shared_buffer.out);
}

int main() {
pthread_t prod_thread, cons_thread;

// Seed random number generator
srand(time(NULL));

// Inisialisasi buffer
init_buffer();

printf("=== SIMULASI PRODUCER-CONSUMER ===\n");
printf("Buffer size: %d item\n", BUFFER_SIZE);
printf("Total item yang akan diproduksi: %d\n\n", MAX_ITEM);

// Tampilkan buffer awal
tampilkan_buffer();

// Buat thread producer dan consumer
pthread_create(&prod_thread, NULL, producer, NULL);
pthread_create(&cons_thread, NULL, consumer, NULL);

// Tunggu kedua thread selesai
pthread_join(prod_thread, NULL);
pthread_join(cons_thread, NULL);

// Tampilkan buffer akhir
tampilkan_buffer();

printf("=== SIMULASI SELESAI ===\n");

return 0;
}

```

BAB III

PENUTUP

3.1 Kesimpulan

Praktikum ini mengimplementasikan tiga program utama yang mendemonstrasikan konsep pemrograman paralel dan sinkronisasi dalam sistem operasi. Program kalkulator paralel menunjukkan bagaimana multithreading dapat digunakan untuk menjalankan tugas-tugas independen secara bersamaan. Program penghitungan kata mengungkapkan bahwa untuk tugas yang kecil, overhead pembuatan thread dapat membuat singlethread lebih efisien, namun multithreading unggul untuk pemrosesan data yang besar. Adapun program producer-consumer berhasil mensimulasikan pengelolaan resource bersama dengan aman menggunakan mutex, menekankan pentingnya sinkronisasi dalam lingkungan konkuren. Secara keseluruhan, praktikum ini memberikan pemahaman mendalam tentang cara mengoptimalkan kinerja sistem melalui pemrograman paralel dan manajemen thread yang efektif.